# 面向状态变化的诊断技能复用：SGLang PD 分离推理的 Agent 自动调优

BoTao Cheng
Tencent

**摘要**

预填充与解码（PD）分离的大语言模型服务，需要联合确定阶段资源、并行策略、批处理和状态传输配置。P800 资源池上的历史运行提示，名义相似的配置可能在缓存、初始化和执行条件变化后呈现不同性能与故障行为。本文解析作者已有的 35 条记录，据此提出面向 SGLang 的经验约束诊断技能与 Agent 自动调优设计。技能声明适用条件、有边界的动作、控制变量、备选预期观测、恢复流程及证据来源。语言模型控制器拟选择诊断和参数族，受约束数值优化器选择具体配置。能力与执行检查限制允许的动作，历史经验提供有上下文的建议；条件变化时需本地重放。诊断支持、进程存活、请求完成及 SLO 达标分别记录。本文给出资源与调优成本目标，并设计区分数值搜索、Agent 决策和技能复用的公平对照。历史诊断由人工决定；自主选择及其性能优势尚未评价。现有证据支撑研究动机与执行基础，不构成 Agent 优势结果。

**关键词：**大语言模型推理服务；预填充与解码分离；Agent 自动调优；诊断技能；资源规划。

## 1 引言

大语言模型推理服务的配置涉及资源分配、并行执行、内存管理、批处理和通信。预填充与解码分离（prefill–decode disaggregation，以下简称 PD 分离）允许针对不同阶段分别配置资源，但也引入了显式的请求状态传输。DistServe 研究时延约束下的阶段分配，Splitwise 探索不同阶段的硬件配置，HexGen-2 则联合考虑异构资源与通信 [7, 16, 23]。这些工作说明，应当联合分析推理阶段及其连接关系。因此，PD 部署本身不构成本文的新贡献。

本文关注选择新配置之前的一个决策：*下一步应该获取哪项观测，或执行哪项受控实验？*例如，请求长度分布发生变化后，现有部署不再满足服务目标。增加预填充实例、降低解码准入上限，以及修改后端支持的传输设置，会影响执行流程中的不同环节。同一种汇总时延异常可能对应多种解释。调优器可以对所有选项进行数值搜索，但可能花费大量时间反复部署并未针对当前限制因素的配置。Agent 可以解读多种证据，但解释看似合理，并不等于诊断已获得验证。

使用大语言模型进行系统调优已有较多研究。GPTuner 将领域知识与数值优化结合；STELLAR 分析日志并积累具有上下文的调优规则；SysInsight 将系统内部机制、诊断与实测支持的规则联系起来 [5, 9, 21]。因此，增加工具调用、上下文记忆或规则置信度，并不足以构成创新。本文将研究问题收窄为：在部署变更可能同时影响多个相互连接阶段的 PD 服务中，显式表示诊断实验及其本地重放条件，能否为调优决策提供有价值的结构？

本文拟将诊断技能表示为一个有边界的程序，通过获取观测，对一组阶段相关假设给出支持、反驳或无法判断的结果。控制器依据这些证据确定参数族的搜索优先级。技能还记录证据产生时

的条件；在新上下文中复用时，需要检查兼容性，并可能重新执行探测。该设计将诊断视为可以修正的经验判断，而非反复执行上一次成功调参动作的指令。

拟议系统包含一个语言模型控制器、人工编写的诊断技能库、数值优化器，以及独立执行器与评价器。已有工作包含人工诊断与 skill 执行，以及配置到压测报告的链路。台账中有 34 条 32 卡记录和一条 48 卡记录，条件各异。初始资源池使用八卡 P800 服务器、双路 AMD EPYC 主机 CPU 和 RDMA，并可扩充。AMD 指主机 CPU。这些记录为设计提供动机，但不证明 Agent 自主决策有效。

本文从历史观测建立可检验的方法链条：识别探索性记录中的状态与协议敏感性，定义预算约束下的证据获取和配置选择，再提出受约束的诊断技能、本地复用检查及非排他搜索引导。后续评价将检验其相对数值优化、固定规则控制器和普通 Agent 的价值。历史表征属于初步证据，方法贡献仍需实验建立。

# 2 背景与相关工作

## 2.1 分离式推理服务与配置

PD 服务将提示词处理和后续 token 生成分配到不同模型副本。实际配置空间包括副本数量、各副本的并行策略、运行时限制和支持的传输路径。Mooncake 围绕 KV 状态组织服务，Sarathi-Serve 则提供了基于分块预填充的重要同置服务方案 [1, 17]。因此，公平评价应同时包含充分调优的同置服务和原生 PD 服务。

SGLang 提供语言模型执行系统和运行时 [22]。其当前 PD 文档与 SGLang-Kunlun 仓库提供了相关集成接口 [2, 18]。这些持续变化的资料，并不能证明本地 P800 镜像已支持其中每项功能。本文设计要求使用与部署版本对应的能力清单，不预设非对称张量并行、特定直接内存传输路径或任意副本路由均可用。

近期预印本还研究了 SLO 感知的 PD 资源数量选择，以及阶段与网络的协同自动扩缩容 [8, 11]。因此，可变资源数量和网络感知资源配置不是本文宣称的研究空白。初始规划器离线推荐部署方案，不声称具备自动扩缩容系统的在线控制机制。

NVIDIA 与 Meta 的大规模研究采用模拟并辅以实测探索部署权衡 [14, 15]，不能描述为只依赖穷举真机实验。Revisiting 研究 KV 传输介质、复用及性能能耗影响，TaiChi 在 TTFT/TPOT 目标下联合聚合与分离 [10, 19]。缓存敏感性和多目标探索因此属于背景，不足以独立构成创新。本文关注实际有状态服务中的诊断动作选择与复用验证。

## 2.2 数值调优与知识引导调优

SCOOT 将受约束贝叶斯优化用于推理引擎调优，并处理已知和隐式配置约束 [4]，是数值搜索的重要直接参照。GPTuner 提取调优知识，用于参数选择与搜索空间优化；$\lambda$-Tune 生成完整数据库配置，并考虑候选配置评价开销 [6, 9]。这些工作要求本文与有竞争力的搜索方法及知识引导方法比较，不能只与默认配置比较。

SLO-Guard 研究考虑失败情况且预算口径一致的推理服务调优；ECO 则将结构化先验与受约束优化结合，在测量预算有限时调优分离式服务 [13, 24]。本文引用版本均为预印本。因此，失败成本计算、模型辅助搜索及有限调优预算，都不能独立作为本文的创新点。

### 2.3 Agent、诊断与知识积累

STELLAR 已支持自适应分析、配置执行和带有调优上下文的规则积累，其规则综合还处理相互冲突的建议 [5]。SysInsight 从代码推导假设、从观测归纳上下文规则，并依据反馈维护可靠性 [21]。预印本 StorageXTuner 将 Agent 角色、探索与实测验证的经验结合 [12]。这些机制与泛化的“Agent 加技能”方案有明显重合，不能将其描述为缺少上下文、反馈或验证。

本文拟研究的区别，是对诊断干预建立显式动作表示：允许改变或观察什么、能够区分哪些竞争性的阶段解释，以及迁移其调优建议之前需要什么证据。这种表示是否超越针对特定应用重新实现已有机制，必须通过实验回答。因此，同工具 Agent 基线和同条件固定规则控制器是必要对照。

### 2.4 测量与比较方法

自动调优比较需要计算实际耗时、随机性与评价开销 [20]。近期预印本 AutoTuneBench 研究 Agent 生成的推理服务及算子测量是否可靠 [3]。本文将独立指标计算和完整成本统计作为实验要求，不将这些保障措施视为主要创新。

## 3 历史观测与研究动机

### 3.1 数据范围与指标语义

数据来源为作者的 35 行 CSV 实验台账，涉及模型标签 `dsv4pro` 和一个具名 session 测试集。R1 至 R35 按数据行编号，不含表头。全部记录标为 PD 分离：34 条为 32 卡 1P1D，R15 为 48 卡 2P1D。因此，它不是完整配比扫描，也没有同置对照。模型标签不足以复现 checkpoint 和精度。镜像、分支列均为空，版本仅在备注中描述。台账指向实验环境中的原始日志，当前分析未独立核查这些日志。

作者确认 P/D 列分别表示 prefill/decode 每分钟 token 吞吐（TPM），总 TPM 包含缓存命中的 token，平均 TPOT 单位为毫秒。正吞吐记录中，prefill TPM 占总 TPM 的 97.68% 至 99.19%。因此总 TPM 是包含复用的服务指标，不等于新执行的预填充计算量或输出生成速度。R32 总 TPM 为 935.65 万，decode TPM 为 131154.61，约 2185.91 token/秒。缓存层归属、加权方式及精确窗口仍需原脚本与日志确认。

23 条记录总 TPM 为正，其中包含降级运行；12 条为零，涉及部署失败、运行失败或缺完整指标的缓存探测。零值不能统一解释为实测零吞吐，零时延字段也不能解释为瞬时执行。汇总数据无法恢复尾时延、联合 SLO 达标或独立运行的不确定性。

### 3.2 缓存上下文与资源权衡

表 1 比较三条 32 卡 1P1D、88 并发的历史记录。R29 标注为冷态单遍，R26、R32 则长时间重复 session 负载。R29 至 R32 的总 TPM 描述性差异为 +45.29%，平均 TTFT 为 −75.74%，但平均 TPOT 增加 21.53%。缓存状态、时长、实际输出和请求过程均不同，不能归为单项优化的效果。部署远端缓存后端，也不能证明每次命中来自哪一层。

R11 与 R15 呈现另一种目标冲突。32 卡 1P1D 到 48 卡 2P1D 的总 TPM 差异为 +35.80%，单卡 TPM 为 −9.47%，平均 TTFT 为 −66.17%，平均 TPOT 为 +22.16%（图 1）。机器组和运

| 记录与协议 | 命中率 (%) | 总 TPM ($10^6$) | 平均 TTFT (秒) | 平均 TPOT (毫秒) |
|---|---|---|---|---|
| R29：冷态单遍 | 89.71 | 6.4401 | 14.9021 | 26.268 |
| R26：2 小时重复 | 98.43 | 9.2725 | 3.7549 | 32.431 |
| R32：24 小时重复 | 99.28 | 9.3565 | 3.6146 | 31.924 |

表 1: 历史观测，不是条件匹配的处理效果。协议与缓存状态来自台账描述，仍需原始日志核验。

行不同。这些点支持分别评价资源与服务目标，但不能建立固定资源下的最优点，也不能证明非单调配比曲线。

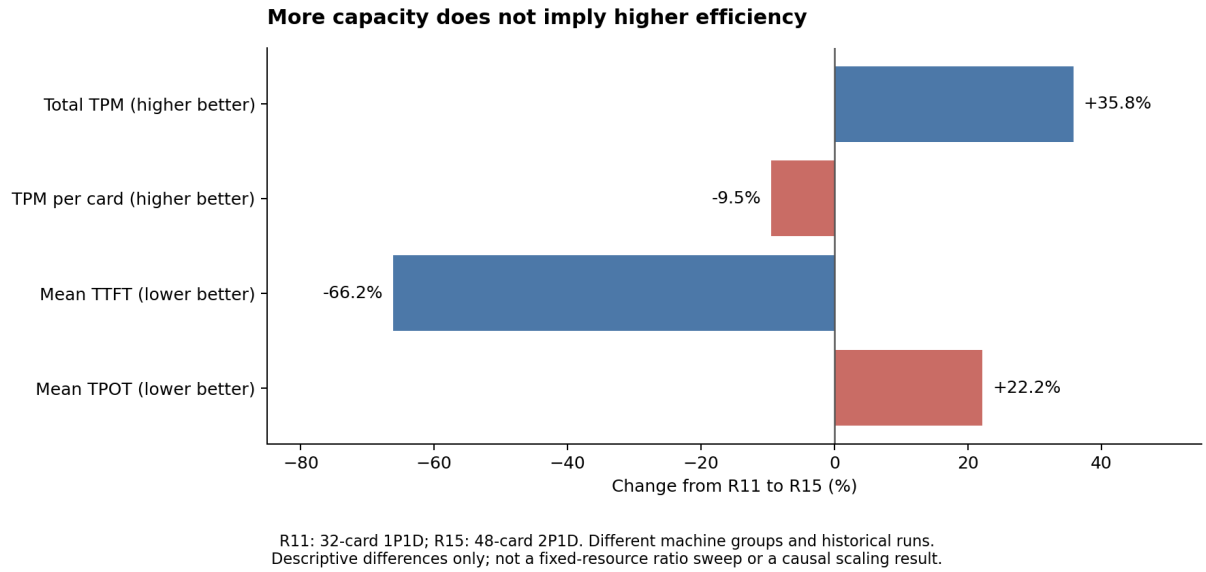



图 1: 历史 R11 至 R15 的描述性变化：总吞吐更高时，单卡吞吐反而更低。资源、机器组及未控制条件不同，不能作因果扩展性解释。图中变化率无量纲。

### 3.3 故障证据与人工决策来源

备注记录了归因变化：R12 将失败关联至 `max_run96`，R13 报告重启状态不兼容并撤回此前归因，R14 在全量重启后仍记录不稳定。这支持保留可修正解释，而非跨版本永久禁用某个值。R16 至 R21 记载了不同干预下的挂死、缓存加载与 KV 传输症状；它们提示竞争解释，尚未独立证明唯一根因。

R19 报告 1800 秒运行完成且无崩溃，但成功请求仅 992/14461（6.86%）。进程存活不等于服务可用。R32 报告 284733/284738 完成，约 99.998244%，包含 5 个失败。其无崩溃观测没有覆盖 R35 在空闲 5.5 小时后的恢复路径。重复同一批 session 也不等于独立负载泛化。

上述诊断由作者决定，skill 执行所选操作。R34 记录配置、资源会话准备、pipeline 和报告链路，完成数为 1578/1606（98.2565%）。它是执行基础，不是 Agent 自主结果。历史案例可用于人工编写初始技能库，但需与最终留出评价分开。包括 R19 在内的统计窗口核对及根因判断，均需原日志支持后才能形成正式机制结论。

# 4 问题定义

## 4.1 任务、配置与实际能力

将任务定义为 $\tau = (m, w, h, v, \rho, \Delta, \mathcal{S})$。其中，$m$ 标识模型与固定精度，$w$ 为请求过程，$h$ 为拓扑，$v$ 为软件栈，$\rho$ 为初始化与状态协议，$\Delta$ 为评价时间范围，$\mathcal{S}$ 为服务目标。协议记录清缓存或复用、部署代次、空闲暴露、session 顺序和预热边界。这些属于控制条件，不代表假定内部状态均可观测。负载特征包括实际输入输出长度分布及到达波动。最大输出长度不能代替实测分布。

配置表示为

$$c = (n_P, n_D, p_P, p_D, \pi, b_P, b_D, k), \tag{1}$$

其中，$n_P, n_D$ 为副本数量，$p_P, p_D$ 为后端支持的并行策略，$\pi$ 为工作进程到资源的映射，$b_P, b_D$ 为运行时限制，$k$ 包含后端支持的状态传输设置。合法配置集合 $\mathcal{C}_\tau$ 由能力清单、显存可行性和资源可用性共同确定。同一比较中的模型与 KV 精度保持固定。这里的数量表示完整模型副本，而非任意切分模型权重所得的部分。

令 $A(c)$ 为分配的加速卡数量；当整机分配决定实际可回收资源时，另行记录节点数。本文不假定每个副本使用相同资源，也不假定某个模型能在特定数量的 P800 上运行。结构性配置变更可能需要重新部署。首版系统定位为离线规划器，不包含在线弹性伸缩或运行中请求迁移。

## 4.2 服务可行性与优化目标

对请求 $i$，定义

$$z_i(c) = \mathbf{1}\{\text{complete}_i \wedge f_i \le \theta_f \wedge o_i \le \theta_o\}, \tag{2}$$

其中，$f_i$ 为客户端观测的首 token 时延（TTFT），$o_i$ 为该请求输出 token 之间的平均间隔。主要负载至少包含两个输出 token；更短的完成请求将依据预先规定的适用性规则单独报告。失败、超时和被拒绝的请求仍计入提交请求总数这一分母。

对同一到达批次的 $N$ 个请求，联合达标率估计为

$$\widehat{a}(c) = \frac{1}{N} \sum_{i=1}^{N} z_i(c). \tag{3}$$

在稳定提交到达率 $\lambda$ 下，$\lambda \widehat{a}(c)$ 是达标请求到达率的估计。本文将其与按完成窗口统计的吞吐量区分，并报告请求到达、观测和队列排空区间。需要检查队列稳定性，并采用固定超时策略，防止较短测量窗口掩盖过载。各项时延分别满足边际尾部目标，并不意味着具有相同的联合达标率。

容量规划目标为

$$\min_{c \in \mathcal{C}_\tau} A(c) \quad \text{满足} \quad a(c) \ge \alpha, \quad \text{队列稳定}. \tag{4}$$

系统返回已找到的实测可行配置中资源最少者，不给出全局最优保证。在相同资源层级内，次级目标为最大化达标服务能力。另设调优目标，最小化达到预先规定的可行性能目标所需成本。这些目标分别报告，不使用任意加权分数掩盖资源消耗与时延。

## 4.3 预算约束下的证据获取

在步骤 $t$，控制器从读取已有证据、执行诊断技能、评价配置及停止中选择动作 $a_t$。状态为 $s_t = (\tau, c_t, \mathcal{D}_t, \mathcal{K}_t, \mathcal{H}_t, B_t)$，包括观测、技能库、可修正的假设和剩余预算。$\mathcal{H}_t$ 不要求包含经过校

准的概率。语言模型的自报置信度不是瓶颈概率的测量值。

每个动作均计入实际耗时 $T(a_t)$，以及资源成本

$$C_A(a_t) = \int_{\text{动作区间}} A_t(u)\, du. \tag{5}$$

诊断、模型加载、重启、预热、恢复及控制器推理均计入成本。并行实验需遵守统一资源上限，或使用相同的资源分配时长预算；所选口径必须在各方法之间保持一致。

# 5 拟议方法

## 5.1 阶段感知证据，而非固定时延相加

控制器使用阶段依赖图；在可以观测时，图中包括客户端提交、请求准入、预填充执行、状态就绪、状态传输、解码准入和 token 发出。该图描述具体部署的实现，不假定普遍适用的时延加和公式。传输与计算可能重叠。首个流式 token 的产生位置和转发路径取决于后端；未跟踪实际首 token 路径之前，不能直接将解码准入等待或 KV 传输时间计入 TTFT。

证据记录包括事件语义、时钟域、单位、样本量及缺失情况。跨节点时长需要使用已表征误差的同步时钟，或改为在单一时钟域内测量。语言模型接收记录引用和结构化摘要。不可用计数器标记为未知，不用看似合理的估计替代。诊断插桩开销单独测量，避免将其无声计入常规推理服务。

## 5.2 将诊断技能表示为可执行实验

定义技能为

$$\sigma = (P, O, H, I, V, F, R, E, G), \tag{6}$$

其中，$P$ 指定适用性，$O$ 为必要观测，$H$ 为解释集合，$I$ 为有边界的观测或干预，$V$ 将结果分为支持、反驳或无法判断，$F$ 为参数族，$R$ 为恢复流程，$E$ 为带来源的证据，$G$ 为外部检查的约束契约。初始技能由工程人员编写。历史备注提供候选条件与提醒，不自动成为经过验证的规则或因果标签。

技能将诊断结果与调优收益分开。一项探测可能支持传输争用假设，但没有改善端到端有效吞吐；反过来，一个配置也可能改善目标，而其声称的原因仍未得到证实。两类观测不能自动互相验证。因此，记录中分别保存假设证据、服务可行性、资源使用和目标变化。

初始技能库拟覆盖预填充压力、解码容量或准入压力、PD 分配失衡，以及传输与计算相互影响。这些解释并非互斥，可能同时成立。每项技能必须说明固定什么、改变什么、是否需要重新部署，以及哪些情况会导致结果无法解释。缓存状态、精度、提交负载或总资源的临时变化，不能被包装成某个参数的孤立效应。

## 5.3 历史经验约束下的技能执行

拟议契约区分四类检查：

$$G = (G_{\text{cap}}, G_{\text{exec}}, G_{\text{eval}}, G_{\text{reuse}}). \tag{7}$$

能力检查将操作限制在已部署的能力清单内；执行检查要求参数类型、资源预留、干预边界、恢复和预算限制；评价检查固定模型精度、负载协议、指标公式及 SLO 分母；复用检查将建议绑定

到版本化状态上下文，必要时要求补充观测或重放。前置条件缺失时，返回明确的无法判断或不可执行结果，不能由语言模型暗中补齐。

| 约束 | 历史动机 | 执行器的拟议行为 |
|---|---|---|
| 实际能力 | 版本描述不完整 | 拒绝不支持的操作，记录未知影响 |
| 执行与状态 | R12 至 R14 的重启歧义 | 核对进程代次，采用已声明恢复协议 |
| 评价口径 | R19 存活但完成率低 | 分开记录存活、完成与联合 SLO 达标 |
| 经验复用 | R32 长稳与 R35 空闲恢复 | 重新检查缓存、空闲、版本及拓扑变化 |

表 2: 由历史问题提出的技能约束，尚不代表控制器已实现或验证。

所有控制器共享相同的必要合法性与测量检查；固定规则基线获得相同技能和初始证据。否则，收益可能只是操作保障不一致，而非 Agent 决策。历史运行时参数告警只作为局部上下文建议；只有独立确认的能力约束才能硬性排除合法空间中的配置。

重启一致性说明了两者的区别：实验可对指定配置变更要求 P/D 协同重启，但经后端确认兼容的局部重启仍可按能力清单执行。不能由一次历史失败导出普遍重启规则。同样，关闭异步加载后不再崩溃，也不能直接认定调优成功，仍需单独测量服务可行性与成本。

每次调用记录约束判定、输入与上下文哈希、来源运行、预期可区分观测、结果、目标变化、恢复情况和全部成本。历史证据在核验前标为人工决策、日志未核验。后续反驳追加其范围与原因，不覆盖原观测。JSON 产物描述这些契约，但没有可执行后端绑定。

### 5.4 适用性检查与本地重放

复用包含三层检查。首先，以可执行兼容性谓词检查技能所需的观测和动作。缺少必要能力时，拒绝该动作。若软件或模型变化的影响未知，则将技能在新上下文中标记为尚未验证；这不一定意味着诊断过程本身不合法。

其次，将当前上下文与证据记录中的上下文比较。负载与规模特征采用在源任务上固定的变换进行归一化，再用透明的距离与覆盖规则识别外推情况。阈值在源验证任务上选择，并在留出测试前固定。相似度只辅助检索，不保证结果正确。

最后，对过期、证据不足或超出覆盖范围的建议，执行其诊断动作的本地重放。重放可以重新支持预期、反驳预期，或仍然无法判断。无法判断不能算作负面结果。证据按任务和上下文保存，因此在某一拓扑下失效，不会抹去其他条件下仍成立的观测。重新验证减少缺乏支持的迁移，但不能普遍保证技能有用。

### 5.5 诊断动作的选择

语言模型提出数量受限的技能标识符列表，并附证据引用、竞争性解释及预期可区分观测。它输出结构化决策，不输出不受约束的脚本，也不能自行认定诊断成立。确定性检查器验证提案是否使用可用观测和合法干预。重复或不能区分现有解释的提案退回修改。

初始策略优先读取已收集证据，再选择预期能以较低成本区分未决解释的诊断。候选评分为

$$Q(d) = \frac{U(d)}{\widehat{C}(d) + \varepsilon}, \tag{8}$$




图 2: 拟议控制流程。图中描述设计，不代表已完成实现。技能证据更新保留原始运行来源。

其中，$U(d)$ 估计对当前解释的区分能力，$\widehat{C}(d)$ 估计所选统计口径下的成本。在获得经过校准的预测器之前，$U(d)$ 采用明确的工程等级评分，允许取“未知”；不将其宣传为期望信息增益或经过校准的因果置信度。评分规则、成本估计器及同分处理策略将在比较前固定。语言模型生成提案的价值，将与使用相同条件的固定规则选择器比较。

诊断证据更新得到支持、被反驳和仍未解决的预期集合。系统允许多个瓶颈同时存在。当现有探测不能区分解释时，控制器退回更广泛的数值探索。该退路很有必要：设计不假设每类负载都存在低成本且正确的诊断。

### 5.6 不永久排除配置的数值搜索引导

获得支持的诊断用于提高某个参数族的优先级，并建议合法初始化点。结构性提案包括副本数量和工作进程放置；在选定结构内部，数值优化器选择受支持的运行时参数。首版实现将使用一种明确记录的受约束优化器，并保留未经本文机制修改的版本作为基线。

除独立确定的能力约束之外，技能只影响提案优先级，不决定硬性可行性。具体而言，保留比例 $\epsilon > 0$ 的提案机会用于完整合法空间：

$$q_t(c) = (1-\epsilon)q_t^{\text{guided}}(c) + \epsilon q_t^{\text{global}}(c). \tag{9}$$

提案分布和 $\epsilon$ 的取值将在实现说明中给出。有限预算下，全局采样不能证明收敛；其目的是减少错误诊断带来的不可逆排除。固定的无改进规则可以触发重新诊断或增加全局探索，并记录额外成本。

最终接受的配置必须经过测量。预测器或 Agent 生成的解释，都不能使未测试配置成为最终推荐。最终评价冻结配置并使用独立请求，探索阶段的测量不能替代留出服务验证。

### 5.7 证据更新与停止

技能记录保留正面、负面和无法判断的结果、上下文特征、配置变化及原始运行标识。即使展示经验支持频率，也应说明它取决于已经执行的干预，不能解释为因果概率。首版不从少量调优运行中学习不受约束的因果模型。

预算耗尽、达到预定目标并经过独立确认，或满足已声明终止规则时停止。Agent 可以提出停止请求，但由执行器检查规则。如果没有测得可行配置，输出应明确说明这一结果和已测试范围，不能暗中放宽 SLO 或更换模型。

# 6 集成计划与算法

## 6.1 SGLang 与 P800 的适配边界

适配器拟提供能力发现、部署、观测收集、诊断执行、压测执行、恢复及配置冻结。逻辑参数先经部署版本检查，再转换为具体启动参数。控制器运行在 token 生成热路径之外，不作为逐 token 调度器。

能力清单记录模型与分词器标识、权重与 KV 类型、运行时与插件修订版本、注意力实现、并行配置、图执行与缓存模式、传输后端，以及支持的路由方式。实际盘点 CPU、XPU、PCIe 和网卡拓扑，不能从 CPU 插槽数推断。未知的单卡显存、网卡速率及传输行为，在测量前保持未填写。公开插件支持情况不能替代本地镜像的验收测试。

部署操作同时包含资源预留和恢复方案。诊断探测在指定实验资源上运行，所有压测解析和目标计算都在 Agent 可编辑决策之外完成。Agent 只能选择已注册操作，不能在比较过程中修改评价输入、指标公式或基线配置。这属于实验控制，不另行作为安全性贡献。

## 6.2 拟议算法

**输入**：任务、合法资源、能力清单、初始技能库与预算。

1. 测量预先声明的初始配置，创建证据记录。
2. 在预算尚未耗尽时循环：
   a. 检索兼容的候选技能及上下文记录。
   b. 请求控制器输出结构化诊断提案。
   c. 检查约束和状态，读取证据或重放诊断。
   d. 记录证据、来源和服务结果。
   e. 通过引导搜索与全局搜索提出合法配置。
   f. 部署、预热、测量，必要时恢复；计算全部成本。
   g. 更新优化器及对应上下文的技能记录。
   h. 检查预先声明的停止或重新诊断规则。
3. 冻结已测得的最佳可行候选，并进行独立验证。

**输出**：配置与证据；或报告未测得可行配置。

以上是拟议自主流程。已有 skill 执行人工选择的任务；完整 Agent 循环、检查器、优化器设置、上下文阈值和停止规则仍需集成与评价。历史工具执行不代表这些拟议组件已经实现。

# 7 实验评价计划

## 7.1 研究问题

RQ1：相较受约束数值搜索和普通工具型 Agent，本文方法能否更快达到预定可行目标？RQ2：相较拥有相同技能的固定规则控制器，诊断选择是否改变了有效的配置决策？RQ3：本地重放与适用性检查是否改善技能向留出上下文的迁移？RQ4：在不同部署规模下，最终推荐的资源消耗与达标服务能力表现如何？

### 7.2 环境、任务与数据划分

起始环境为四台八卡 P800 服务器、双路 AMD EPYC 9K84 CPU；作者报告 96 核规格与 RDMA 支持。历史 R15 已在另一机器组使用 48 卡。仍需盘点单路或整机核数口径、NUMA、显存、网卡及准确版本。资源池可扩充，但不预设未测量的大规模布局合法。

计划固定当前 checkpoint 与精度，并在可行时增加第二种稳定模型配置。历史备注描述 16 卡预填充副本（PP2，每阶段 TP8）和 16 卡解码副本。经核验且后端支持后，可比较 48 卡的 2P1D/1P2D，以及 64 卡的 3P1D/2P2D/1P3D，并保留 32 卡 1P1D。这些是计划布局，不是实验结果。不支持的布局标为不可行，不能在同一比较中更换精度或模型。

任务包含短输入短输出、长输入短输出、短输入长输出，以及混合长度请求过程。服务容量研究采用开环到达，固定并发压测作为辅助诊断。源任务用于积累技能证据；随后冻结上下文阈值和初始技能库，再测试留出的模型、负载或规模任务。冷启动、带经验启动，以及可选的任务内更新分别报告。

35 条历史记录属于开发证据，不是最终测试集。人工选择可用于编写技能，不能证明自主选择准确率。后续运行前固定源任务、验证与测试划分、相同初始知识、提示词、优化器及缓存重启空闲协议。先回收关键案例的 trace 和版本清单，再重复仍未解决的测量。持续满压与空闲恢复分别作为测试条件。

### 7.3 基线与组件归因

| 方法 | 共同条件 | 比较目的 |
|---|---|---|
| 已调优原生服务 | 同置或分块预填充及原生 PD；固定模型和精度 | 端到端工程参照 |
| 受约束优化器 | 相同合法空间、预算及测量流程 | 检验超出数值搜索的收益 |
| 固定规则控制器 | 相同技能、观测与初始知识 | 检验语言模型决策的必要性 |
| 工具型 Agent | 相同模型与工具；普通反馈和日志检索 | 检验超出通用 Agent 编排的收益 |
| 无条件技能复用 | 同库；保留必要检查，不重验历史建议 | 检验错误迁移的代价 |
| 完整方案 | 包含全部已声明组件 | 检验联合效果及开销 |

表 3: 计划开展的对照。借鉴既有系统的改编实现必须标明为改编，不能写成运行了原始实现。

借鉴 STELLAR 或 SysInsight 的源码感知或规则积累 Agent，是重要的强基线。如果原始实现无法控制 SGLang，应准确描述适配方式，并向其提供相同证据与工具。不能将其他硬件上已报道的加速比直接作为本文的定量基线。消融实验将分别移除主动诊断、以原始日志检索替换技能记录、禁用适用性重放，以及移除全局探索，每次只改变一个机制。

### 7.4 指标、不确定性与资源核算

主要指标包括达到预定目标所需的资源分配时长与实际耗时、联合 SLO 达标率，以及相同提交负载下最终实测资源占用。次要指标包括输出 token 吞吐、实际输入输出长度、请求级 TTFT 与 TPOT、token 级相邻 token 时延、失败配置、重新部署和诊断动作。P99 结果需要充分样本量和明确的不确定性。只有在解释受控或经独立确认的场景，才报告诊断准确率。

计划对核心服务配置点至少进行三次独立重复；控制器与搜索使用多个随机种子，在预算允许时初始设置为五个。运行顺序随机化或交错安排，配对比较使用相同的负载实现。一次过载运行中的请求，不能视为整个调优过程的独立重复。未达到目标的运行作为删失结果报告，不能在计算达标耗时时将其丢弃。

最终评价前固定阈值、模型选择、搜索范围、停止条件、Agent 提示词、生成设置和 token 预算。Agent 推理使用独立容量，或明确测量其造成的干扰。并行搜索共享相同资源上限，或按相同资源分配时长预算比较。离线建库与后续调优成本分别报告；经验复用的摊销收益需要计算，不能直接假定。

### 7.5 机制测试与决策后果

若后端暴露适当操作，首项机制测试比较独立传输、独立解码及二者受控并发执行。另一项比较阶段容量估计给出的推荐与端到端配置排名。这些测试用来确认平台上是否存在本文假设的诊断歧义，不能自动证明存在新的系统机制。

对每次成功诊断，追踪完整决策链：已有观测、考虑的备选解释、所选探测、探测结果、搜索优先级变化、最终配置及独立验证。对错误诊断，报告浪费成本，以及重放或全局探索是否帮助恢复。该分析将技能机制与可测结果连接起来，避免将解释质量当作性能收益的替代指标。

## 8 后续 Agent 结果与证据要求

第 3 节已有历史观测，但尚无条件匹配的自主控制器结果。本节规定评价拟议方法所需的证据，不能将人工决策结果归于 Agent。主结果将以配对比较及不确定性估计回答 RQ1 至 RQ4。需要的图包括：达标耗时与预算累积下的性能曲线、资源与服务能力前沿、诊断结果及决策后果、有无重放时的留出任务技能迁移，以及开销和失败动作分解。本稿不包含虚构的加速比曲线。

| 证据 | 所需分析 | 当前状态 |
|---|---|---|
| E1：机制 | 竞争性解释与受控诊断结果 | 尚未测量 |
| E2：调优 | 同预算基线及达到目标的耗时 | 尚未测量 |
| E3：迁移 | 冻结源技能库后的留出上下文测试 | 尚未测量 |
| E4：容量 | 同负载资源占用及同资源服务能力 | 尚未测量 |
| E5：消融 | 固定规则、普通 Agent、重放及探索对照 | 尚未测量 |

表 4: 证据补全清单。表中为进度标记，不是实验数据。

负面发现必须改变论文主张。若固定规则与语言模型控制器相当，就不能将性能优势归因于 Agent 推理。若重放成本超过节省成本，就需要收窄复用的适用范围。若不存在有意义的阶段诊断歧义，就应修改诊断设计。只比默认参数更好，不能证明方法贡献。

## 9 局限与有效性威胁

主要局限是尚无经过验证的自主集成与同条件方法评价。历史证据为异质、汇总且由人工决策的记录；版本缺失和原日志未核验限制机制结论。与 Agent 存储及数据库调优系统的差异仍是假设。既有系统已有上下文和冲突处理，因此受约束诊断动作必须展示超越规则改名的实际价值。

单一加速卡后端限制硬件泛化能力。增加资源数量可以扩大规模覆盖，但不能证明方法可以移植至其他加速卡系列。观测不足可能妨碍可靠诊断，诊断插桩也可能扰动执行。上下文特征和阈值可能对源任务过拟合。即使只编辑一个配置字段，动态资源共享、请求长度不确定性、缓存状态和网络波动仍可能混淆干预结果。

语言模型控制器可能遗漏重要解释、虚构参数语义，或选择成本过高的探测。注册动作、证据引用及独立检查能够限制这些行为，但不能证明它们不会发生。调优耗时减少不意味着稳态容量增加；反过来，资源更节省的最终部署可能需要更高调优成本，两者都必须报告。

首版范围不包含自动算子重写、模型质量权衡、跨厂商 KV 互操作及生产在线迁移。优先采用公开模型和可复现的负载描述；实际代码、机器细节与 trace 的披露取决于组织允许的发表范围。无法提供的材料应明确列出，不能写成已经公开发布。

## 10 结论

本文将 35 条 P800 历史实验记录与 SGLang PD 服务的经验约束诊断技能设计联系起来。这些记录为上下文感知评价、可修正解释，以及存活、完成和 SLO 分离记录提供动机。核心假设是，Agent 可以使用有边界的技能和经本地重验的经验，以可接受的完整调优成本改善配置决策。在声称方法优势之前，仍需完成自主集成和公平对照。

## A 实验前必须固定的实现决策

最终实验前，需明确：模型与精度标识；SGLang、插件及传输组件版本；逻辑动作到后端操作的映射；控制器模型与提示词；搜索算法和设置；技能上下文及重放阈值；成本核算口径；到达与排空区间语义；SLO 和超时；源任务、验证任务和测试任务划分；随机种子；停止条件。这些是尚待确定的实现决策，不是省略未报的实验发现。

## B 主张与证据对应关系

| 候选主张 | 最低证据要求 |
| --- | --- |
| 诊断选择改善调优决策 | 同工具固定规则与 Agent 对照；决策变化关联留出任务性能 |
| 适用性重放减少负迁移 | 冻结源技能库；上下文变化任务；无条件复用及成本消融 |
| 资源推荐具有较高效率 | 相同负载与 SLO；实测可行资源比较；稳定性与不确定性分析 |
| 方法可在 P800 上实际使用 | 版本化集成产物；验收测试；完整开销与失败记录 |

# C 历史数据与决策来源

台账文件为 `experiments/dsv4pro_32card_pd_ratio_scan.csv`，SHA-256 为：

`e688fa3dabfea4dab60d4fab8eb32b69c0cd0f551adb1ca9344036602429dfaa`。

R1 至 R35 是从 1 开始的数据行号。分析脚本保留源值，增加行号、可用性标记和算术检查，不将缺失零值转换为成功观测。作者确认了 TPM 语义、TPOT 单位和人工诊断来源。CSV 指向原始日志，但当前分析使用台账，尚未完成日志审计。备注中的修复与故障归因不作为本文新修复算法或已验证因果标签。后续 Agent 运行使用新运行标识及明确决策来源，不将旧记录改标为自主实验。

# 参考文献


[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 117–134. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[2] Baidu Baige Contributors. SGLang-Kunlun: Hardware platform plugin. Project repository, 2026. URL https://github.com/baidu-baige/sglang-kunlun. Accessed 2026-10-18; deployed version not yet supplied.

[3] Li Chen. AutoTuneBench: Trustworthy measurement for agent auto-tuning of LLM serving engines. arXiv:2609.18123, 2026. URL https://arxiv.org/abs/2609.18123. Preprint.

[4] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. SCOOT: SLO-oriented performance tuning for LLM inference engines. In *Proceedings of the ACM Web Conference 2025*, pages 829–839, 2025. doi: 10.1145/3696410.3714930. URL https://arxiv.org/html/2408.04323v2.

[5] Chris Egersdoerfer, Philip Carns, Shane Snyder, Robert Ross, and Dong Dai. STELLAR: Storage tuning engine leveraging LLM autonomous reasoning for high performance parallel file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025. doi: 10.1145/3712285.3759887. URL https://arxiv.org/html/2602.23220v1.

[6] Victor Giannakouris and Immanuel Trummer. $\lambda$-tune: Harnessing large language models for automated database system tuning. *Proceedings of the ACM on Management of Data*, 3(1), 2025. doi: 10.1145/3709652. URL https://2025.sigmodconf.hosting.acm.org/toc-3-1.html.

[7] Youhe Jiang, Ran Yan, and Binhang Yuan. HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/pdf?id=Cs6MrbFuMq.

[8] Ruiqi Lai, Hongrui Liu, Chengzhi Lu, Zonghao Liu, Siyu Cao, Siyang Shao, Yixin Zhang, Luo Mai, and Dmitrii Ustiugov. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025. URL https://arxiv.org/abs/2512.03416. Preprint.

[9] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. GPTuner: A manual-reading database tuning system via GPT-guided bayesian optimization. *Proceedings of the VLDB Endowment*, 17(8):1939–1952, 2024. doi: 10.14778/3659437.3659449. URL https://vldb.org/pvldb/vol17/p1939-tang.pdf.

[10] Jiaxi Li, Yue Zhu, Eun Kyung Lee, and Klara Nahrstedt. Revisiting disaggregated large language model serving for performance and energy implications. arXiv:2601.08833, 2026. URL https://arxiv.org/abs/2601.08833. Technical-report version supplied in the research materials.

[11] Luchang Li, Dongfang Li, Bozhao Gong, and Yu Zhang. SLO-aware compute resource allocation for prefill-decode disaggregated LLM inference. arXiv:2603.04716, 2026. URL https://arxiv.org/abs/2603.04716. Preprint.

[12] Qi Lin, Zhenyu Zhang, Viraj Thakkar, Zhenjie Sun, Mai Zheng, and Zhichao Cao. StorageXTuner: An LLM agent-driven automatic tuning framework for heterogeneous storage systems. arXiv:2510.25017, 2025. URL https://arxiv.org/abs/2510.25017. Preprint.

[13] Christian Lysenstøen. SLO-Guard: Crash-aware, budget-consistent autotuning for SLO-constrained LLM serving. arXiv:2604.17627, 2026. URL https://arxiv.org/abs/2604.17627. Preprint.

[14] Meta Inference Team. Optimizing deployment configurations for LLM inference: Challenges and insights. In *Proceedings of the 9th MLSys Conference, Industry Track*, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/97dc07f1253ab33ee514f395a82fa7cc-Abstract-Conference.html.

[15] Tiyasa Mitra, Ritika Borkar, Nidhi Bhatia, et al. A pragmatic exploration of prefill-decode disaggregation in large scale inference. In *Proceedings of the 9th MLSys Conference, Industry Track*, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/d49cee5f3a79d97d719df255689d83d7-Abstract-Conference.html.

[16] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture*, 2024. URL https://www.microsoft.com/en-us/research/publication/splitwise-efficient-generative-llm-inference-using-phase-splitting/.

[17] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. arXiv:2407.00079, 2024. URL https://arxiv.org/abs/2407.00079. Cited version is the technical report, not the later journal version.

[18] SGLang Contributors. PD disaggregation. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/pd_disaggregation. Accessed 2026-10-18; a release or commit must be pinned for experiments.

[19] Chao Wang, Pengfei Zuo, Zhangyu Chen, Yunkai Liang, Zhou Yu, and Ming-Chang Yang. Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized LLM serving. arXiv:2508.01989, 2025. URL https://arxiv.org/abs/2508.01989. Cited technical-report version.

[20] Floris-Jan Willemsen, Richard Schoonhoven, Jiří Filipovič, Jacob O. Tørring, Rob van Nieuwpoort, and Ben van Werkhoven. A methodology for comparing optimization algorithms for auto-tuning. *Future Generation Computer Systems*, 159:489–504, 2024. doi: 10.1016/j.future.2024.05.021. URL https://ac.liacs.nl/blog/methodology-paper.

[21] Xinyi Zhang, Tiantian Chen, Zhentao Han, Zhaoyan Hong, Wei Lu, Sheng Wang, Mo Sha, Anni Wang, Shuang Liu, Yakun Zhang, Feifei Li, and Xiaoyong Du. Why database manuals are not enough: Efficient and reliable configuration tuning for DBMSs via code-driven LLM agents. *Proceedings of the VLDB Endowment*, 19(6):1358–1371, 2026. doi: 10.14778/3797919.3797940. URL https://arxiv.org/html/2603.22708v1.

[22] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

[23] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 193–210. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[24] Qingyun Zou, Bin Gao, Zhuobin Huang, Weng-Fai Wong, Tulika Mitra, and Bingsheng He. ECO: Energy-oriented configuration optimization for attention FFN disaggregated LLM serving. arXiv:2610.08373, 2026. URL https://arxiv.org/abs/2610.08373. Preprint.